Faculty of Sciences

Department of Computer Sciences

Master thesis

Procedural Generation of Solvable Cooperative Levels for the Laser Learning Environment

Author:
Charels Hugo
hugo.charels@ulb.be

Supervisors:
Lenaerts Tom
Molinghen Yannick

Academic year 2025-2026

Contents

1 Introduction ... 1
1.1 Context ... 1
1.2 Motivation ... 1
1.3 Problem Statement ... 1
1.4 Contributions ... 2
1.5 Thesis Structure ... 2
2 Background ... 4
2.1 Multi-Agent Reinforcement Learning ... 4
2.2 The Laser Learning Environment ... 4
2.2.1 Structure and Mechanics ... 4
2.2.2 An Illustrative Example ... 5
2.2.3 Relevance as a Benchmark ... 5
2.3 Procedural Content Generation ... 6
2.4 Boolean Satisfiability ... 6
3 Related Work ... 7
4 Methods ... 8
4.1 Problem Formalization ... 8
4.2 Solver by Reduction to SAT ... 10
4.2.1 Definition of Sets ... 10
4.2.2 Definition of Variables ... 11
4.2.3 Constraints and Logical Encoding ... 11
4.2.4 Clause Complexity and Polynomial Size ... 14
4.2.5 Correctness of the Reduction ... 15
4.2.6 Complexity-Theoretic Consequences ... 16
4.3 Cooperation Detection ... 16
4.3.1 Strict SAT Encoding ... 17
4.3.2 Why This Captures Cooperation ... 17
4.3.3 Formal Theorem and Proof ... 17
4.3.4 Practical Algorithm ... 18
4.4 Level Generators ... 18
4.4.1 Design Pattern ... 18
4.4.2 Generation Targets ... 18
4.4.3 Random Solvable Generator ... 19
4.4.4 Constrained Random Solvable Generator ... 19
4.4.5 Random Cooperative Generator ... 19
4.4.6 Constrained Random Cooperative Generator ... 20
4.4.7 Constructive Solvable Generator ... 20
4.4.8 Constructive Cooperative Generator ... 21
4.4.9 Summary ... 21
4.5 Benchmarking Setup ... 21
4.5.1 Metrics ... 21
4.5.2 Solver and Level Sets ... 22
4.5.3 Protocol ... 22
4.5.4 Outputs ... 22
5 Experiments ... 23
5.1 Experimental Setup ... 23
5.1.1 Test Levels ... 24

5.2 Results 24
5.2.1 Clause Count 25
5.2.2 Solving Time 25
5.3 Discussion 26
6 Conclusion 27
6.1 Summary 27
6.2 Limitations 27
6.3 Future Work 27
Bibliography 28

Chapter 1

Introduction

1.1 Context

Cooperative Multi-Agent Reinforcement Learning (MARL) studies settings in which several agents must learn behaviours that succeed only through coordination. In such settings, the environment is not merely a benchmark container: its structure determines which coordination patterns are possible, which ones are necessary, and how difficult they are to discover.

This dependence on environment design is especially strong in sparse-reward cooperative tasks. When reward is issued only after a full joint objective has been achieved, a training level is useful only if it exposes a meaningful coordination challenge while remaining actually solvable. Levels that are unsolvable, trivial, or solvable without genuine inter-agent dependence provide poor training signal and limit what the agents can learn.

1.2 Motivation

Procedural Content Generation (PCG) offers a way to scale environment design, but only if the generated instances satisfy the properties that matter for training. In the present setting, two properties are central.

First, a level must be **solvable**: the agents must admit at least one valid joint execution that reaches the exits. Second, a level should **require cooperation**: success should depend on at least one agent enabling another agent's progress, rather than on several agents independently solving parallel subproblems.

This thesis studies these questions in the Laser Learning Environment (LLE) [1], a 2D cooperative benchmark whose laser mechanics create explicit inter-agent dependencies. LLE is a useful case study because its grid structure is simple enough to formalise, yet rich enough to express non-trivial cooperative bottlenecks.

1.3 Problem Statement

The central problem this thesis addresses is the following: how can we automatically generate LLE levels that are provably solvable and provably require inter-agent cooperation?

We target two formal properties and one broader design objective:

1. **Solvability** - the level admits at least one valid joint action sequence through which the agents collectively occupy all exit tiles. This is a necessary baseline: a level that cannot be completed is useless for training.
2. **Cooperation requirement** - the level cannot be solved without at least one agent performing a cooperative act, namely using same-colour occupancy to make a teammate's path traversable. This property ensures that the level is not trivially solvable by independent agent behaviour.
3. **Learnability** - the generated instances should expose coordination patterns that are plausible training signals for MARL algorithms. Unlike solvability and cooperation, learnability is not formalised or certified in this thesis; we treat it as a design objective and discuss it in terms of structural affordances that may facilitate exploration.

The approach we adopt to address the first two properties is a reduction to Boolean Satisfiability (SAT): we encode the constraints of an LLE level as a propositional formula in conjunctive normal form (CNF), and delegate the decision procedure to a modern SAT solver. The generation algorithms then use these SAT checks as acceptance criteria.

1.4 Contributions

This thesis makes the following contributions:

- **A SAT-based solver for bounded-horizon LLE solvability.** We provide a CNF encoding of the LLE decision problem over a bounded time horizon. The solver either returns a satisfying assignment encoding a valid joint trajectory or certifies that no such trajectory exists within the chosen horizon ().
- **A formal cooperation detector.** We define a strict variant of the LLE beam semantics in which agents can no longer use same-colour occupancy to truncate their own beams. We show that a level requires this blocking-based cooperative action if and only if the standard encoding is satisfiable and the strict encoding is unsatisfiable ().
- **A family of procedural level generators.** Building on the solver and cooperation detector, we implement six generators: random solvable, constrained random solvable, random cooperative, constrained random cooperative, constructive solvable, and constructive cooperative. Each accepted level is certified by the solver to satisfy the advertised properties of its output ().
- **An empirical comparison of two SAT formulations.** We compare two alternative encodings of the agent-uniqueness constraint on four benchmark levels, measuring their effect on CNF size, model generation time, and solver runtime (,).

The resulting framework is specific to LLE at the encoding level, but the broader methodology is more general: formalise the desired level properties, decide them with a solver, and place that verifier inside the generation loop.

1.5 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 introduces only the background needed for the rest of the thesis. Chapter 3 positions the work relative to prior literature. Chapter 4 presents the formalisation, SAT reduction, cooperation detector, generators, and benchmarking protocol. Chapter 5 reports the experimental results. Chapter 6 concludes.

Chapter 2

Background

2.1 Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) extends Reinforcement Learning to settings in which $n \geq 2$ agents act simultaneously within a shared environment. A standard formalisation is the stochastic game introduced by Shapley [2] and widely used as a foundation for MARL [3]. The model is given by the tuple $\left(X, \{U^i\}_{i=1}^n, T, \{R^i\}_{i=1}^n, \gamma\right)$, where U^i is the action space of agent i and R^i is its reward function. At each time step, every agent selects an action independently, the joint action $\boldsymbol{u}_t = (u_t^1, ..., u_t^n)$ determines the state transition, and each agent receives a reward.

This thesis concerns the fully cooperative setting, where all agents share a single reward signal: $R^1 = R^2 = ... = R^n = R$. The difficulty in this setting is not only to find good individual actions, but to discover joint behaviour that is useful at the team level. In sparse-reward tasks, supportive actions may carry no immediate local benefit, which makes them difficult to learn from experience alone.

2.2 The Laser Learning Environment

The Laser Learning Environment (LLE) [1] is a 2D grid-based cooperative puzzle game designed as a benchmark for MARL algorithms. It supports between one and four agents and is built around a set of mechanics that make cooperative coordination both necessary and difficult to learn.

2.2.1 Structure and Mechanics

An LLE level is played on a rectangular grid of height H and width W. Each cell is either a floor tile, a wall, a laser source, or an exit. The key components are as follows.

Agents. Each agent is assigned a unique colour from the set $C = \{0, 1, ..., n_a - 1\}$. At the start of an episode, agent c is placed at its designated starting cell $s(c)$. At each time step, an agent may stay in place or move to an adjacent non-wall cell. Agents cannot occupy the same cell simultaneously.

Lasers. Each laser source $(c, d, p) \in \mathcal{S}$ emits a beam of colour c in direction $d \in \{N, S, E, W\}$ starting from cell p. The beam propagates cell by cell in direction d until it is blocked by a wall or by an agent of the same colour. Any agent of a different colour that enters a traversed cell is eliminated.

This creates a direct dependency between colours: each beam can be neutralised only by the matching agent.

Exits. A level contains exactly n_a exit tiles. The episode is completed successfully when all exit tiles are occupied simultaneously. Since agents cannot share positions, this means that each exit is occupied by exactly one agent in a winning configuration.

Zero-incentive bottlenecks. Intermediate cooperative acts, such as blocking a beam for a teammate, carry no immediate reward. The reward is issued only when all agents complete the level. An agent may therefore need to take a locally useless action that is beneficial only to the team, which makes exploration difficult [1].

The full LLE implementation contains additional mechanics that are not modelled in this thesis, such as gems and void tiles. We omit them because they are not needed for the bounded solvability questions studied here. In particular, void tiles can be conservatively replaced by walls when constructing instances for the current solver.

2.2.2 An Illustrative Example

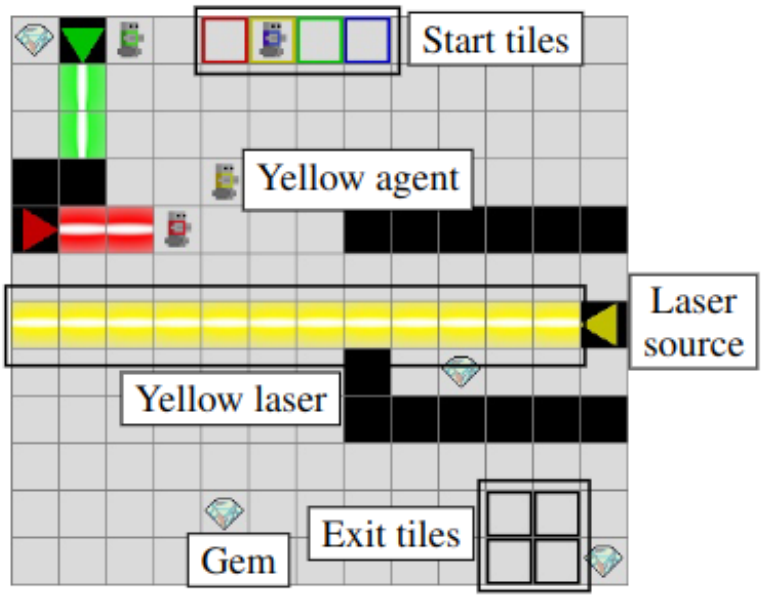

Figure 1: An annotated LLE level. Laser beams (coloured lines) block movement for agents of a different colour. The yellow agent blocks the yellow laser to allow the other agents to pass.

The figure above illustrates a typical LLE dependency. One agent must occupy the path of its own laser so that the beam is truncated and another agent can pass. This is the type of blocking-based cooperation studied throughout the rest of the thesis.

2.2.3 Relevance as a Benchmark

LLE has been used to evaluate several value-based MARL algorithms, including methods augmented with prioritised experience replay, n-step returns, and intrinsic curiosity via random network distillation [1]. Despite these enhancements, agents frequently fail to escape coordination bottlenecks in LLE, even when they perform well on more conventional cooperative benchmarks.

For that reason, LLE is a suitable benchmark for studying whether formally constrained generation can produce levels that are both solvable and structurally cooperative.

2.3 Procedural Content Generation

Procedural Content Generation (PCG) refers to the algorithmic creation of content such as levels, maps, or environments with limited direct human input [4]. In the MARL setting, PCG is valuable because it can supply a large and diverse stream of training instances instead of relying on a small fixed set of hand-crafted levels.

PCG methods are commonly grouped into three families. **Constructive methods** generate content in a single forward pass and are typically fast, but offer no formal guarantees. **Search-based methods** treat generation as an optimisation problem guided by a fitness function [5]. They can express rich objectives, but they inherit the limitations of the chosen heuristic. **Constraint-based methods** encode desired properties explicitly and use a solver to find content satisfying them.

This thesis adopts the constraint-based paradigm because the target properties are logical rather than purely stylistic: we want accepted levels to come with formal guarantees of solvability and cooperation.

2.4 Boolean Satisfiability

The Boolean Satisfiability problem (SAT) is the canonical NP-complete decision problem. Given a propositional formula φ over a set of Boolean variables $x_1, \ldots, x_n$, the question is: does there exist a truth assignment $\nu : \{x_1, \ldots, x_n\} \to \{\text{true}, \text{false}\}$ such that $\varphi(\nu) = \text{true}$? The central role of SAT in complexity theory was established by the Cook-Levin theorem [6].

SAT solvers typically operate on formulas in **Conjunctive Normal Form** (CNF), where a formula is represented as a conjunction of clauses and each clause is a disjunction of literals:

$$\varphi = \bigwedge_{i=1}^{m} \bigvee_{j} \ell_{i,j}$$

Any propositional formula can be converted to an equisatisfiable CNF formula in polynomial time. When a CNF formula is satisfiable, a solver returns a satisfying assignment; otherwise it returns UNSAT.

Modern SAT solvers, especially CDCL-based solvers, often perform very well on large structured instances in practice. This empirical effectiveness is what makes SAT a plausible backend for the solvability and cooperation checks developed in this thesis.

Chapter 3

Related Work

Chapter 4

Methods

4.1 Problem Formalization

We now define the semantic objects and decision problems used throughout the methods chapter. This section operates at the level of the game semantics; the SAT encoding of these objects follows in .

The LLE environment supports between 1 and 4 agents. Accordingly, throughout this thesis we assume that $1 \leq n_a \leq 4$.

Definition 4.1 (LLE Level)

An LLE level is a tuple $L = (H, W, C, s, \mathcal{W}, \mathcal{S}, \mathcal{E})$ where:

- $H, W \in \mathbb{N}^+$ are the height and width of the grid.
- $P = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$ is the set of all grid positions.
- $C = \{0, 1, ..., n_a - 1\}$ is the set of agent colours, with $1 \leq n_a \leq 4$.
- $s : C \to P$ assigns an initial position to each agent, and s is injective.
- $D = \{N, S, E, W\}$ is the set of laser directions.
- $\mathcal{W} \subseteq P$ is the set of wall positions.
- $\mathcal{S} \subseteq C \times D \times P$ is the set of laser sources. A source $(c, d, p) \in \mathcal{S}$ emits a laser of colour c in direction d from position p.
- $\mathcal{E} \subseteq P$ is the set of exit positions, with $|\mathcal{E}| = |C|$.
- In the instances studied in this thesis, each colour appears in at most one laser source.
- The sets $s(C)$, $\mathcal{W}$, $\mathcal{E}$, and

  $$P_{\text{src}} = \{p \in P \mid \exists c \in C, d \in D : (c, d, p) \in \mathcal{S}\}$$

  are pairwise disjoint.

We also write

$$C_{\text{src}} = \{c \in C \mid \exists d \in D, p \in P : (c, d, p) \in \mathcal{S}\}$$

for the set of colours that actually have a source.

Definition 4.2 (Active Beams)

Fix a time step t and a joint position map $p_t : C \to P$.

For a source $(c, d, p_s) \in \mathcal{S}$, consider the grid ray starting at p_s and extending in direction d.

- Under the **standard beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall or the first cell occupied by agent c.
- Under the **strict beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall.

A laser of colour c is active at a position when the beam of the unique source of colour c is active there.

Definition 4.3 (Valid Joint Trajectory)

A **horizon** $T_{\max} \in \mathbb{N}$ is the maximum number of joint moves allowed in a candidate solution.

A **joint trajectory** of length $T_{\max}$ is a sequence of joint positions $\sigma = \left(p_0, p_1, \ldots, p_{T_{\max}}\right)$ where each $p_t : C \to P$ assigns a position to every agent at time step t.

A joint trajectory is **valid** if it satisfies the following conditions:

- **Initialization:** $p_0(c) = s(c)$ for all $c \in C$.
- **Movement:** For all $t < T_{\max}$ and all $c \in C$, $p_{t+1}(c)$ is reachable from $p_{t(c)}$ in one step: the agent may stay in place or move to a 4-neighbouring cell, but it may not move into a wall or a laser-source cell.
- **No collision:** $p_{t(c_1)} \neq p_{t(c_2)}$ for all t and all distinct $c_1, c_2 \in C$.
- **Conservative hand-off rule:** For all $t < T_{\max}$ and all distinct $c_1, c_2 \in C$, agents may not enter a cell occupied by another agent at the previous time step; that is, $p_{t+1}(c_1) \neq p_{t(c_2)}$ and $p_{t(c_1)} \neq p_{t+1}(c_2)$.
- **Laser safety:** No agent c_1 occupies a cell at time t where a laser of colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is active under the standard beam semantics of Definition 4.2.
- **Stay on exit:** If $p_{t(c)} \in \mathcal{E}$ for some $t < T_{\max}$, then $p_{t+1}(c) = p_{t(c)}$.

Definition 4.4 (Solvability)

A level L is **solvable** with horizon $T_{\max}$ if there exists a valid joint trajectory σ of length $T_{\max}$ such that the set of occupied positions at time $T_{\max}$ is exactly the set of exits:

$$\left\{p_{T_{\max}}(c) \mid c \in C\right\} = \mathcal{E}$$

A level is **solvable** without qualification if it is solvable for some finite horizon.

The restriction to a bounded horizon is natural for the SAT encoding. In the model studied here, beam activity at time t is a deterministic function of the joint position map p_t. Hence the full state is determined by the joint agent positions. If a trajectory repeats the same joint position twice, the intervening segment forms a loop and can be removed without affecting reachability of later states. Therefore, if a level is solvable at all, it is solvable within a finite horizon bounded by the number of collision-free joint configurations.

Definition 4.5 (Bounded-Horizon LLE Solvability Problem)

The **bounded-horizon LLE solvability problem** is the following decision problem.

- **Input:** an LLE level L and a horizon $T_{\max} \in \mathbb{N}$.
- **Question:** does L admit a valid joint trajectory of length $T_{\max}$ whose final occupied positions are exactly the exits?

Definition 4.6 (Strict Beam Semantics)

A **strict trajectory** of length $T_{\max}$ is defined exactly as in Definition 4.3, except that beam activity is computed using the strict beam semantics of Definition 4.2: same-colour occupancy no longer truncates the corresponding beam.

In the model studied here, the relevant cooperative action is precisely this same-colour beam-truncation mechanism: an agent occupies a position that would otherwise allow its own beam to continue, thereby opening a path for another agent.

Definition 4.7 (Cooperation Requirement with Horizon $T_{\max}$)

A level L is said to **require cooperation** with horizon $T_{\max}$ if:

- L is solvable under the standard semantics.
- L admits no strict trajectory of length $T_{\max}$ whose final positions occupy all exit tiles.

When the horizon is clear from context, we simply say that L requires cooperation. This bounded form is the one used throughout the rest of the methods chapter, since both the SAT solver and the cooperation detector operate at a fixed finite horizon.

4.2 Solver by Reduction to SAT

4.2.1 Definition of Sets

We reuse the level objects introduced in : the grid dimensions H and W, the position set P, the colour set C, the direction set D, the wall set $\mathcal{W}$, the source set $\mathcal{S}$, the exit set $\mathcal{E}$, and the start map s.

The SAT reduction introduces a finite horizon $T_{\max} \in \mathbb{N}$, which is the maximum number of joint moves allowed in the bounded decision problem, together with the discrete time-step set $T = \{0, 1, \ldots, T_{\max}\}$.

We also define

$$P_{\text{src}} = \{p \in P \mid \exists c \in C, d \in D : (c, d, p) \in \mathcal{S}\}$$

for the set of source positions.

We further write

$$C_{\text{src}} = \{c \in C \mid \exists d \in D, p \in P : (c, d, p) \in \mathcal{S}\}$$

for the set of colours that actually have a source.

To match the benchmark and generated instances studied in this thesis, we assume that each colour appears in at most one laser source. Under this assumption, when a source of colour c exists, its position and direction are uniquely determined by c.

We also write $\text{start}(c) \in P$ for the initial position of agent $c \in C$.

4.2.2 Definition of Variables

We introduce three families of propositional variables.

- $a_{c,x,y,t}$: true iff agent $c \in C$ occupies position $(x, y) \in P$ at time step $t \in T$.
- $b_{c,d,x,y,t}$: true iff the beam emitted by the source $(c, d, p_s) \in \mathcal{S}$ is active at position $(x, y) \in P$ at time $t \in T$.
- $l_{c,x,y,t}$: true iff a laser of colour $c \in C_{\text{src}}$ is active at position $(x, y) \in P$ at time $t \in T$.

4.2.3 Constraints and Logical Encoding

We now formalise the constraint families used in the reduction. Each group of clauses corresponds to one logical component of the bounded-horizon decision problem.

1. **Initialization**

   - **Agents:**
     Each agent $c \in C$ is placed at its designated starting position $\text{start}(c)$ at $t = 0$; all other positions are unoccupied by that agent:

     $$\bigwedge_{c \in C} \bigwedge_{(x,y) \in P} \begin{cases} a_{c,x,y,0} & \text{if } (x, y) = \text{start}(c) \\ \neg a_{c,x,y,0} & \text{otherwise} \end{cases}$$

   - **Laser sources:**
     For each laser source $(c, d, (x_s, y_s)) \in \mathcal{S}$, the beam is always active at its origin, at every time step:

     $$\bigwedge_{(c,d,(x_s,y_s)) \in \mathcal{S}} \bigwedge_{t \in T} b_{c,d,x_s,y_s,t}$$

2. **Agent Movements**

   We define the set of positions reachable from (x, y) in one step as (x, y) itself together with its four grid neighbours, excluding walls and laser-source positions:

   $$\text{next}(x, y) = \{(x', y') \in \{(x, y), (x, y-1), (x+1, y), (x, y+1), (x-1, y)\} \mid (x', y') \in P, (x', y') \notin \mathcal{W}, (x', y') \notin P_{\text{src}}\}$$

   - **Forward consistency:**
     If agent c is at position (x, y) at time t, it must be at some position in $\text{next}(x, y)$ at time $t + 1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t} \rightarrow \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t+1}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} \neg a_{c,x,y,t} \vee \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t+1}$$

Two methods are proposed to enforce that each agent occupies at most one position at each time step.

- **Global uniqueness:**
  No two distinct positions can both be occupied by agent c at time t:

$$\bigwedge_{c\in C}\bigwedge_{t\in T}\bigwedge_{(x_1,y_1)\in P}\bigwedge_{\substack{(x_2,y_2)\in P,\\ (x_2,y_2)\neq(x_1,y_1)}} \neg a_{c,x_1,y_1,t} \vee \neg a_{c,x_2,y_2,t}$$

- **Local uniqueness:**
  No two distinct positions in $\text{next}(x, y)$ can simultaneously be occupied by agent c at time $t + 1$:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P}\bigwedge_{(x',y')\in \text{ next}(x,y)}\bigwedge_{\substack{(x'',y'')\in \text{ next}(x,y),\\ (x'',y'')\neq(x',y')}} \neg a_{c,x',y',t+1} \vee \neg a_{c,x'',y'',t+1}$$

  Backward consistency:
  If agent c is at position (x, y) at time $t + 1$, it must have been at some position in $\text{next}(x, y)$ at time t:

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} a_{c,x,y,t+1} \rightarrow \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t}$$

$$\Updownarrow$$

$$\bigwedge_{c\in C}\bigwedge_{t=0}^{T_{\max}-1}\bigwedge_{(x,y)\in P} \neg a_{c,x,y,t+1} \vee \bigvee_{(x',y')\in \text{ next}(x,y)} a_{c,x',y',t}$$

- **No simultaneous occupation:**
  Two distinct agents cannot share the same position at the same time, nor can they move to a position already occupied by the other agent:

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C, c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t\in T} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C, c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t+1} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1\in C}\bigwedge_{c_2\in C, c_2\neq c_1}\bigwedge_{(x,y)\in P}\bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t+1}$$

- **Victory condition:**
  Each exit must be occupied by at least one agent at time $T_{\max}$. Since $|\mathcal{E}| = n_a$ and agents cannot share positions, this enforces a bijection between agents and exits:

$$\bigwedge_{(x,y)\in\mathcal{E}} \bigvee_{c\in C} a_{c,x,y,T_{\max}}$$

- **Stay on exit:**
  Once an agent reaches an exit, it remains there for all subsequent time steps:

$$\bigwedge_{c\in C} \bigwedge_{(x,y)\in\mathcal{E}} \bigwedge_{t=0}^{T_{\max}-1} \neg a_{c,x,y,t} \vee a_{c,x,y,t+1}$$

3. **Laser Activity**

We define $\text{next}_{d(x,y)}$ as the position immediately adjacent to (x, y) in direction d:

$$\text{next}_{d(x,y)} = \begin{cases} (x, y-1) \text{ if } d = N \\ (x+1, y) \text{ if } d = E \\ (x, y+1) \text{ if } d = S \\ (x-1, y) \text{ if } d = W \end{cases}$$

- **Walls block beams:**
  A beam cannot be active at a wall position:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in\mathcal{W}} \bigwedge_{t\in T} \neg b_{c,d,x,y,t}$$

- **Beam propagation:**
  Beam propagation clauses are instantiated only when the successor cell $(x', y') = \text{next}_{d(x,y)}$ lies inside the grid, is not a wall, and is not itself a source cell. Source cells are handled separately by the initialization clauses above. Under these conditions, the beam is active at (x', y') iff it is active at (x, y) and no agent of colour c occupies (x', y'):

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right)$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} \neg b_{c,d,x,y,t} \vee a_{c,x',y',t} \vee b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} b_{c,d,x,y,t} \vee \neg b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} \neg a_{c,x',y',t} \vee \neg b_{c,d,x',y',t}$$

- **Link between beam and laser variables:**
  Since each colour has at most one source in the instances considered here, the laser variable $l_{c,x,y,t}$ is true at a position iff the beam of the unique source of colour c is active there:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} b_{c,d,x,y,t} \leftrightarrow l_{c,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \left(\neg b_{c,d,x,y,t} \vee l_{c,x,y,t}\right) \wedge \left(\neg l_{c,x,y,t} \vee b_{c,d,x,y,t}\right)$$

- **Agents cannot step on active lasers:**
  An agent of colour c_1 cannot occupy a position where an active laser of some source colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is present. Agents are immune only to lasers of their own colour:

$$\bigwedge_{c_1\in C} \bigwedge_{\substack{c_2\in C_{\text{src}},\\ c_2\neq c_1}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} l_{c_2,x,y,t} \rightarrow \neg a_{c_1,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{c_1\in C} \bigwedge_{\substack{c_2\in C_{\text{src}},\\ c_2\neq c_1}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \neg l_{c_2,x,y,t} \vee \neg a_{c_1,x,y,t}$$

4.2.4 Clause Complexity and Polynomial Size

To show that the reduction has polynomial size, it is enough to bound the number of generated clauses as a function of the input parameters. Let

$$\begin{aligned} n &= |C|, \\ p &= |P| = HW, \\ s &= |\mathcal{S}|, \\ e &= |\mathcal{E}|, \\ \tau &= T_{\max} + 1 \end{aligned}$$

and write

$$V = P \setminus (\mathcal{W} \cup P_{\text{src}})$$

for the walkable cells.

We count clauses rather than literals. The main families admit the following bounds.

- **Initialization.** The agent-initialisation clauses contribute exactly np unit clauses, and the laser-source initialisation contributes exactly $s\tau$ unit clauses.
- **Movement constraints.** Forward consistency contributes at most $n(\tau-1)p$ clauses. The two uniqueness formulations differ here:

$$\text{global uniqueness} = n(\tau-1)\binom{p}{2}$$

$$\text{local uniqueness} = n(\tau - 1) \sum_{u \in V} \binom{|\text{next}(u)|}{2}$$

Since $|\text{next}(u)| \leq 5$, the local uniqueness term is bounded by

$$n(\tau - 1)10\ |V| \leq 10n(\tau - 1)p$$

and backward consistency contributes at most $n(\tau - 1)p$ further clauses. The collision and conservative hand-off clauses contribute at most

$$\binom{n}{2}(\tau + 2(\tau - 1))p = \binom{n}{2}(3\tau - 2)p$$

The exit condition contributes exactly e clauses, and the stay-on-exit condition contributes exactly $ne(\tau - 1)$ clauses.

- **Laser constraints.** The laser-safety clauses contribute at most $ns\tau p$ clauses. Beam propagation contributes at most $3s\tau p$ clauses, because each admissible propagation edge yields either one wall-blocking clause or three equivalence clauses. The beam/laser linking clauses contribute exactly $2s\tau p$ clauses.

Therefore the total number of clauses is polynomial in n, p, s, and τ. More precisely, with the global formulation the dominant term is

$$O(n\tau p^2 + n^2\tau p + ns\tau p)$$

while with the local formulation it is

$$O(n\tau p + n^2\tau p + ns\tau p)$$

Since each clause is generated by a simple bounded computation inside these loops, the reduction itself is computable in polynomial time as well. This justifies the claim that bounded-horizon LLE solvability is polynomial-time reducible to SAT.

4.2.5 Correctness of the Reduction

Proposition 4.8 (Correctness of the SAT Reduction)

Let L be an LLE level and let $T_{\max}$ be a horizon. For either movement formulation described above, the CNF formula $\Phi(L, T_{\max})$ is satisfiable if and only if there exists a valid joint trajectory of length $T_{\max}$ for L.

Proof.

For soundness, assume $\Phi(L, T_{\max})$ is satisfiable. Initialization fixes exactly one start position for each agent at time 0. For the global formulation, forward consistency together with pairwise exclusion ensures by induction on time that each agent occupies exactly one legal position at every later step. For the local formulation, the same conclusion follows from forward consistency, local exclusivity, and backward consistency. We may therefore define a joint trajectory $p_{t(c)}$ by reading off the unique true agent variable for each colour c and time t.

The movement clauses enforce legal motion between consecutive steps; the collision clauses enforce both simultaneous separation and the conservative hand-off rule; the laser clauses enforce safety with respect to active beams; and the exit clauses enforce the terminal condition. Hence the extracted trajectory is valid.

For completeness, assume a valid joint trajectory of length $T_{\max}$ is given. Set each $a_{c,x,y,t}$ according to whether agent c occupies (x, y) at time t in the trajectory. Set beam variables $b_{c,d,x,y,t}$ and laser variables $l_{c,x,y,t}$ according to the deterministic beam dynamics induced by the same agent positions. Every clause family is then satisfied by construction: initialization matches the start state, movement and collision clauses match the trajectory semantics, beam propagation follows the induced laser dynamics, and the final positions occupy all exits. Therefore $\Phi(L, T_{\max})$ is satisfiable. $\square$

4.2.6 Complexity-Theoretic Consequences

We can now state the consequence for the decision problem introduced in Definition 4.5.

The bounded-horizon LLE solvability problem lies in **NP**. A candidate trajectory can be verified in polynomial time by simulating the joint execution and checking that all agents occupy the exit tiles at the end without violating the movement, collision, and laser constraints defined in .

Combined with the polynomial-time construction above and Proposition 4.8, this shows that bounded-horizon LLE solvability is polynomial-time many-one reducible to SAT:

$$\text{LLE-Solvability} \leq_p \text{SAT}$$

Thus bounded-horizon LLE solvability is **at most as hard as SAT**: any SAT algorithm yields an algorithm for this decision problem with only polynomial overhead from the reduction.

Whether bounded-horizon LLE solvability is also **NP-hard** remains open in the present work. Establishing NP-hardness would require a polynomial-time reduction in the opposite direction, from a known NP-hard problem to LLE solvability. This thesis does not claim such a result.

It is also important to distinguish proved statements from standard complexity-theoretic beliefs. The question whether $\mathrm{P} = \mathrm{NP}$ remains open. Accordingly, statements here about worst-case difficulty should be read only through the formal claims we have established: SAT is NP-complete, bounded-horizon LLE solvability belongs to NP, and the reduction above places bounded-horizon LLE solvability within the polynomial-time many-one image of SAT.

In practice, this positioning explains why a SAT-based approach is attractive: the solver inherits the strong empirical performance of modern CDCL SAT solvers on many structured instances, even though the worst-case complexity remains exponential.

4.3 Cooperation Detection

4.3.1 Strict SAT Encoding

Recall from Definition 4.6 that strict beam semantics changes only one aspect of the dynamics: same-colour occupancy no longer truncates the corresponding beam. Same-colour immunity is unchanged.

Accordingly, the strict SAT encoding keeps the standard laser-safety clauses and replaces only the same-colour beam-propagation rule. For every source $(c, d, p_s) \in \mathcal{S}$, every admissible propagation edge from (x, y) to (x', y'), and every time step $t \in T$, the strict encoding uses

$$b_{c,d,x',y',t} \leftrightarrow b_{c,d,x,y,t}$$

instead of the standard equivalence

$$b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right).$$

Thus the beam continues through agents of the matching colour instead of stopping at them. We denote the resulting CNF formula by $\Phi_{\text{strict}}(L, T_{\max})$ and the corresponding solver by StrictSolver.

4.3.2 Why This Captures Cooperation

Under the LLE mechanics studied here, the cooperative action of interest is for an agent to occupy a cell that would otherwise allow its own beam to continue, thereby making another agent's path safe. Standard solvability allows this beam-truncation mechanism; strict solvability removes it.

Therefore, if a level is solvable under the standard semantics but unsatisfiable under the strict semantics, every successful standard solution must rely on at least one same-colour beam-truncation step.

4.3.3 Formal Theorem and Proof

Theorem 4.9 (Cooperation Detection Criterion)

Let L be an LLE level and $T_{\max}$ a time horizon. Then L requires cooperation with horizon $T_{\max}$ if and only if $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

Proof.

$(\rightarrow)$ Assume that L requires cooperation with horizon $T_{\max}$. By Definition 4.7, L is solvable under the standard semantics, so $\Phi(L, T_{\max})$ is satisfiable. Suppose for contradiction that $\Phi_{\text{strict}}(L, T_{\max})$ is also satisfiable. Then there exists a strict trajectory whose final positions occupy all exit tiles. Since strict beam semantics differs from the standard one only by removing same-colour beam truncation, such a trajectory is also a valid standard trajectory

that succeeds without using that mechanism. This contradicts Definition 4.7. Therefore $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

($\leftarrow$) Assume that $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable. The first condition implies that L is solvable under the standard semantics. Suppose that some successful standard trajectory used no same-colour beam-truncation step. Then the same joint positions would also satisfy the strict beam semantics, because the only semantic difference between the two models concerns exactly that truncation mechanism. This would yield a satisfying assignment for $\Phi_{\text{strict}}(L, T_{\max})$, contradicting unsatisfiability. Hence every successful standard trajectory must use at least one same-colour beam-truncation step, so L requires cooperation with horizon $T_{\max}$. $\square$

4.3.4 Practical Algorithm

The cooperation detector runs two SAT calls on the same level:

1. Run $\text{Solver}(L, T_{\max})$. If the result is UNSAT, the level is unsolvable for that horizon, so it is rejected before cooperation is considered.
2. Run $\text{StrictSolver}(L, T_{\max})$. If the result is UNSAT, the level requires cooperation for the same horizon.

Both calls share the same bounded horizon and differ only in the beam-propagation clauses. For benchmark levels, the horizon can be chosen from known solution lengths; for generated levels, it is the user-supplied generation parameter $T_{\max}$.

4.4 Level Generators

4.4.1 Design Pattern

All generators follow a common architecture built around three principles:

1. **SAT as oracle**: the solver is not post-hoc; it is embedded in the generation loop. A candidate level is accepted only if the solver confirms the desired property (solvability, cooperation).
2. **Separation of concerns**: level construction and property verification are decoupled. Generators build candidate levels using domain-specific heuristics; the solver decides acceptance.
3. **Extensibility**: every generator extends `BaseGenerator` and is registered via the `@register_generator` decorator, making it available to the CLI without modifying core code.

In implementation terms, each generator repeatedly performs the following loop: sample or construct a candidate layout, reject it if it violates generator-specific structural constraints, build an `lle.World`, and finally run the appropriate SAT-based acceptance test. Solvable generators stop after the first candidate certified satisfiable within the target horizon, while cooperative generators add the strict-beam counterfactual test and, optionally, a cooperation-profile filter.

4.4.2 Generation Targets

Viewed through the solvability and cooperation definitions of , the generator family targets the three level categories shown in Figure Figure 2. Solvable generators accept levels in categories (b) and (c), cooperative generators accept only levels in category (c), and unsolvable levels in category (a) are always rejected.

(a) Unsolvable
rejected by all generators

(b) Solvable, no cooperation
accepted only by solvable generators

(c) Solvable and cooperative
target of cooperative generators

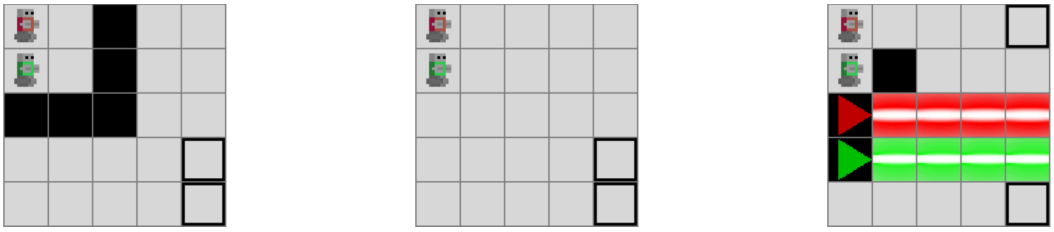

Figure 2: Target level categories for the generator family.

4.4.3 Random Solvable Generator

The random solvable generator is the baseline member of the family. It samples pairwise-distinct positions for agent starts, exits, walls, and laser sources uniformly over the grid, assigns a random direction to each source, and submits the resulting world to the solver. A candidate is accepted only if it is satisfiable within the requested horizon $T_{\max}$; when a lower bound $T_{\min}$ is provided, the generator also requires the candidate to be unsatisfiable for $T_{\min} - 1$, thereby selecting levels that fall inside a difficulty window.

This generator is deliberately simple. Its main value is methodological: it gives an unbiased sampling baseline against which more structured generators can be compared. Its main weakness is rejection rate. As the grid grows and the number of interacting entities increases, purely random layouts quickly become dominated by unsolvable or trivial instances.

4.4.4 Constrained Random Solvable Generator

A structured variant that biases generation toward solvable configurations before any SAT call is made. Relative to the random solvable generator, it rejects candidates that are already geometrically degenerate, for example when a laser points outside the grid immediately, when a laser would have zero beam length, or when an exit lies on an unavoidable beam segment.

These filters do not themselves prove solvability, but they remove a large class of obviously bad candidates before invoking the solver. The generator therefore remains sound with respect to the formal solvability guarantee, while typically spending less time on layouts that fail for purely local geometric reasons.

4.4.5 Random Cooperative Generator

The random cooperative generator extends the random solvable generator with a second SAT test based on the strict semantics of . A candidate is accepted only if it is satisfiable under the standard encoding and unsatisfiable under the strict encoding. This guarantees that every accepted level structurally requires the beam-truncation mechanism identified as cooperation in Definition 4.7.

The current implementation augments this binary guarantee with a **cooperation profile analyzer**. The binary detector remains the formal guarantee used throughout the thesis: a level is cooperative if and only if it is satisfiable under the standard semantics and unsatisfiable under the strict semantics. The analyzer adds a second layer whose purpose is to distinguish **which kind** of cooperation the accepted level exhibits.

Given a cooperative level, we first extract a valid joint plan from the standard SAT model. We then run selective counterfactual checks in which one agent at a time loses the same-colour laser interaction used for helping behaviour. If the level becomes unsatisfiable under that selective restriction, the agent is identified as a necessary helper. By combining these counterfactual checks with the helping actions observed in the extracted plan, we build a directed dependency graph between agents.

This dependency graph is used as a generation target. In the present implementation, the generator can recognise and filter levels according to the following profile families:

- **cooperative**: binary cooperation is required, regardless of finer structure;
- **asymmetric**: at least one one-way helping relation is present;
- **mutual**: two agents depend on each other;
- **chain**: dependencies form a directed chain without branching;
- **distributed**: one agent depends on multiple distinct helpers;
- **fully coupled**: all agents belong to a single strongly connected dependency component.

The important methodological point is that profile control is layered on top of the existing formal machinery. The SAT encodings still certify solvability and binary cooperation. The profile analyzer uses those certified levels as input and acts as a classification and filtering layer for the generator.

4.4.6 Constrained Random Cooperative Generator

The constrained random cooperative generator combines the geometric filters of the constrained solvable generator with the binary cooperation test and optional profile filter of the random cooperative generator. In other words, it first avoids immediately degenerate geometries, then requires the surviving candidates to satisfy the same solver-based cooperation criterion.

This generator therefore targets the same formally certified output class as the random cooperative generator, but with a sampling distribution biased away from trivial failures. It is useful when the goal is not only to obtain cooperative levels, but to obtain them with fewer discarded samples.

4.4.7 Constructive Solvable Generator

The constructive solvable generator replaces blind sampling with a partial-by-construction layout. It reserves one disjoint horizontal or vertical lane per agent, places each start at one end of its lane and the corresponding exit at the other end, and only samples walls and lasers outside the reserved traversable lanes. Additional lasers are accepted only if their beam segments avoid the reserved cells.

The solver still acts as the final verifier, but the sampling process is strongly biased toward jointly solvable instances.

4.4.8 Constructive Cooperative Generator

The constructive cooperative generator further specialises the constructive idea by planting a deliberate dependency pattern. One laser is placed so that a helper agent must truncate a beam of its own colour before another lane becomes traversable. The resulting candidate is then verified with the standard and strict SAT encodings. In the current implementation, this generator supports the `cooperative` and `asymmetric` profile filters. It is useful when one wants deliberately cooperative instances rather than merely sampling for cooperation and hoping to find it by rejection.

4.4.9 Summary

Generator	Construction Bias	Solvable	Cooperative
Random Solvable	Uniform random sampling	Yes (SAT check)	No
Constrained Random Solvable	Random + geometric rejection	Yes (SAT check)	No
Random Cooperative	Uniform random sampling	Yes (SAT check)	Yes (strict UNSAT)
Constrained Random Cooperative	Random + geometric rejection	Yes (SAT check)	Yes (strict UNSAT)
Constructive Solvable	Reserved agent lanes	Yes (SAT check)	No
Constructive Cooperative	Reserved lanes + planted dependency	Yes (SAT check)	Yes (strict UNSAT)

Table 1: Overview of the implemented generators and their guaranteed properties.

4.5 Benchmarking Setup

The first benchmark implemented in this project isolates the SAT solver rather than the generators. Its goal is to compare the two movement formulations used in the encoding: the **local** formulation, which combines neighbourhood-based exclusivity with backward consistency, and the **global** formulation, which enforces uniqueness by pairwise exclusion over the whole grid.

4.5.1 Metrics

For each run, the benchmarking code records:

- the total number of clauses in the generated CNF;
- the clause count contributed by each major constraint family;

- the CNF generation time;
- the SAT solving time;
- the total time obtained by summing generation and solving.

The implementation also stores per-constraint method profiles for the movement constraint, which allows us to inspect the part of the final CNF generated inside the movement-constraint module.

4.5.2 Solver and Level Sets

All benchmark runs use the same SAT backend as the main solver implementation, namely `Minisat22` through the PySAT interface. By default, the benchmarking script can evaluate the six hand-crafted LLE levels listed in `levels.py`, each paired with a horizon known to be sufficient for solvability. It can also benchmark custom levels constructed programmatically with `WorldBuilder`.

The experiment reported in Chapter 5 uses four representative levels: three synthetic instances of increasing size and one original LLE level. This combination exposes both scaling behaviour and the behaviour of the solver on a realistic cooperative puzzle.

4.5.3 Protocol

For each level and each movement formulation, the benchmark performs one profiled run to extract the exact clause counts and the full constraint breakdown. It then repeats the same solver invocation for 100 runs, each time on a fresh copy of the world, and reports the mean and standard deviation of generation time and solve time. Using fresh world copies avoids benchmark contamination by mutable environment state.

No timeout or parallel speedup is introduced in this protocol. The measurements should therefore be read as direct comparisons between the two SAT formulations on the same machine, rather than as hardware-independent absolute performance claims.

4.5.4 Outputs

The benchmarking pipeline produces a console summary table, a JSON file containing aggregated benchmark statistics, and a set of plots for clause counts, per-constraint clause breakdowns, and timing statistics. Chapter 5 draws on these outputs to interpret the trade-off between the two movement formulations.

Chapter 5

Experiments

The goal of this first experimental evaluation is to compare two alternative formulations of the agent uniqueness constraint introduced in , and to measure their effect on the size of the CNF formula and on the time required by the SAT solver across levels of increasing complexity.

5.1 Experimental Setup

This experiment follows the protocol of . All runs use the `Minisat22` backend through PySAT. For each level and each movement formulation, the benchmark records one exact clause profile and then repeats the solve 100 times to obtain mean generation and solve times with standard-deviation error bars.

The analytical comparison is driven by the uniqueness constraint itself. The **global** formulation adds a clause for every unordered pair of positions, yielding a quadratic contribution in $|P|$. The **local** formulation restricts exclusivity to neighbourhood-sized successor sets and supplements it with backward consistency, yielding a linear contribution in $|P|$ because $|\mathrm{next}(x, y)| \leq 5$. The laser and collision clauses are shared by both formulations and therefore do not affect the comparison between them.

More precisely, if we isolate the **method-specific** contribution of the uniqueness mechanism, then for one agent and one time step we obtain:

$$\text{global uniqueness} = \binom{|P|}{2} = \frac{(|P| - 1)\ |P|}{2}$$

$$\text{local uniqueness} + \text{backward consistency} = \sum_{u \in V} \binom{|\mathrm{next}(u)|}{2} + |V|$$

where V denotes the set of walkable cells. Since $|\mathrm{next}(u)| \leq 5$ for every $u \in V$, this local term satisfies

$$\sum_{u \in V} \binom{|\mathrm{next}(u)|}{2} + |V| \leq \left(\binom{5}{2} + 1 \right) |V| \leq 11\ |P|$$

This yields the theoretical crossover used to interpret the plots: the global contribution is smaller on very small instances, but once

$$\binom{|P|}{2} > 11\ |P|$$

the local contribution becomes smaller. The upper-bound crossover occurs at $|P| = 23$, where the two expressions are equal.

5.1.1 Test Levels

Four levels of increasing size and complexity were used:

3x3	**5x5**	**8x8**	**LLE Level 6**
2 agents, 1 laser	3 agents, 2 lasers	4 agents, 3 lasers	4 agents, 3 lasers

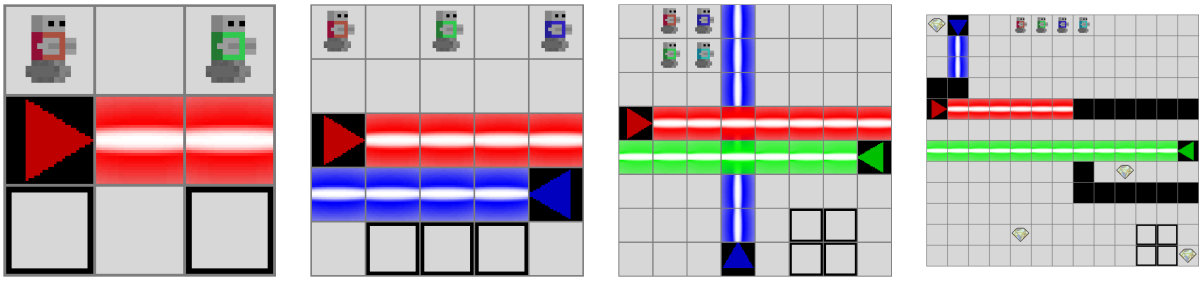

Figure 3: The four test levels used for benchmarking, ordered by increasing complexity.

The first three levels were constructed specifically to test scaling behaviour across grid size and agent count. The fourth is LLE Level 6, one of the hand-crafted levels shipped with the LLE benchmark, included to assess solver performance on a realistic cooperative level.

5.2 Results

5.2.1 Clause Count

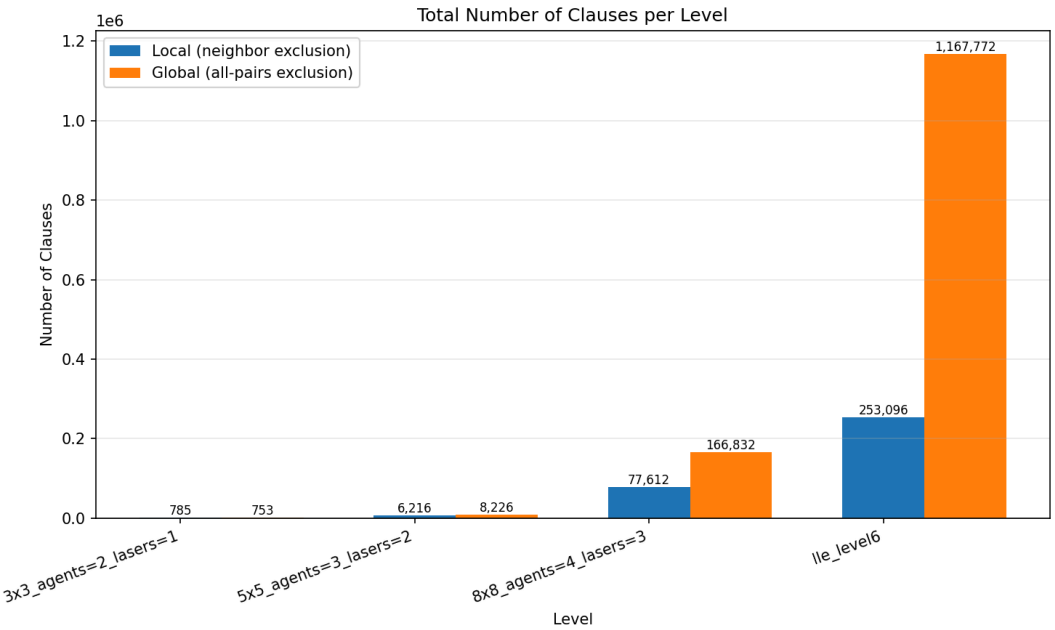

Figure 4: Number of clauses generated by the global and local uniqueness formulations across the four test levels.

The clause-count results closely match the theoretical expectation. On the smallest 3×3 level, the global formulation is slightly more compact than the local one (753 clauses versus 785), which is consistent with the fact that the instance lies below the expected crossover regime. From the 5×5 level onward the trend reverses: the local formulation generates 6,216 clauses versus 8,226 for the global one, then 77,612 versus 166,832 on the 8×8 level, and finally 253,096 versus 1,167,772 on LLE Level 6. The gap widens rapidly as the number of walkable cells and interacting agents increases, which is precisely the behaviour predicted by the linear versus quadratic growth of the two formulations.

5.2.2 Solving Time

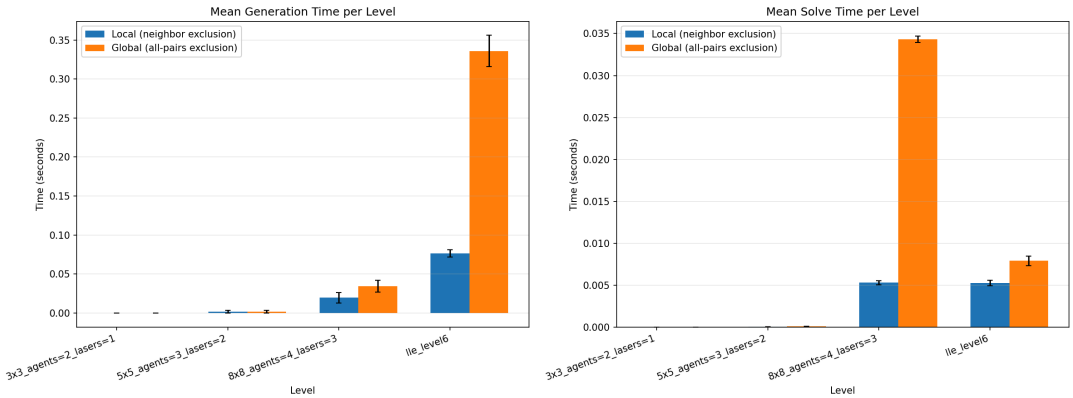

Figure 5: SAT solver time (seconds) for the global and local uniqueness formulations across the four test levels.

The timing results show the same qualitative picture. On the smallest synthetic levels both methods solve essentially instantaneously, and the difference is negligible. Once the instances become moderately large, however, the local formulation is consistently faster both to generate and to solve. The contrast is especially visible on the 8×8 level, where the global formulation has the largest solve-time penalty, and on LLE Level 6, where the cost of generating the much larger global CNF dominates total runtime. In other words, the clause-count reduction is not merely a syntactic improvement; it translates into a measurable performance gain.

5.3 Discussion

The experiment supports a clear design choice. The global formulation can be marginally attractive on very small grids because it introduces fewer clauses there, but this advantage disappears almost immediately. For the levels of practical interest in this thesis, the local formulation is both more compact and faster. We therefore treat the local encoding as the preferred default in the implementation.

The broader experimental agenda remains open. In particular, cooperation-detection cost, generator acceptance rates, and structural diversity of accepted levels should be studied in a dedicated second experimental phase. The current benchmark nonetheless establishes an important baseline: the movement formulation chosen inside the SAT encoding has a first-order effect on both CNF size and runtime, and that effect is already substantial on realistic LLE instances.

Conclusion

6.1 Summary

This thesis developed a SAT-based framework for generating LLE levels with formal guarantees. We first reduced bounded-horizon LLE solvability to satisfiability of a CNF formula, then defined a strict beam-semantics counterfactual that turns cooperation detection into a second SAT query on the same level. On top of these decision procedures, we implemented a family of random, constrained, and constructive generators whose outputs are certified by the solver before acceptance.

The first experimental evaluation compared two movement formulations inside the SAT encoding. The results showed that the local formulation is the better default for the levels studied here: once the instances move beyond the smallest toy case, it produces substantially smaller CNF formulas and lower runtimes than the global all-pairs formulation.

6.2 Limitations

The current framework remains deliberately scoped. The encoding is tailored to the subset of LLE mechanics needed for solvability and cooperation analysis, so features such as gems and void tiles are abstracted away. The guarantees are also horizon-bounded: the solver decides whether a level is solvable within a fixed $T_{\max}$, not whether it is solvable under an unbounded notion of play. Finally, the empirical results reported here cover only the movement-formulation comparison; they do not yet constitute a full evaluation of generator acceptance rates, diversity, or downstream learnability.

6.3 Future Work

Several natural extensions follow from these limitations.

- Extend the experimental section with generator-focused studies: acceptance rates, cooperation profile frequencies, and diversity measures across parameter regimes.
- Generalise the encoding to richer LLE mechanics or to other cooperative grid environments, while preserving the same solver-in-the-loop methodology.
- Investigate whether learnability can be linked to structural properties of the generated levels, for example by combining the current formal guarantees with curriculum design or MARL training experiments.
- Refine the cooperation-profile analysis so that it becomes not only a filtering mechanism for generators, but also an evaluation tool for the qualitative structure of cooperative tasks.

Bibliography

[1] Y. Molinghen, R. Avalos, M. Van Achter, A. Nowé, and T. Lenaerts, "Laser Learning Environment: A New Environment for Coordination-Critical Multi-Agent Tasks," in *Artificial Intelligence and Machine Learning*, F. A. Oliehoek, M. Kok, and S. Verwer, Eds., Cham: Springer Nature Switzerland, 2025, pp. 135–154.

[2] L. S. Shapley, "Stochastic Games," *Proceedings of the National Academy of Sciences*, vol. 39, no. 10, pp. 1095–1100, 1953, doi: 10.1073/pnas.39.10.1095.

[3] M. L. Littman, "Markov Games as a Framework for Multi-Agent Reinforcement Learning," in *Machine Learning Proceedings 1994*, Morgan Kaufmann, 1994, pp. 157–163. doi: 10.1016/B978-1-55860-335-6.50027-1.

[4] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016. doi: 10.1007/978-3-319-42716-4.

[5] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-Based Procedural Content Generation: A Taxonomy and Survey," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011, doi: 10.1109/TCIAIG.2011.2148116.

[6] S. A. Cook, "The Complexity of Theorem-Proving Procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.

[7] M. Davis, G. Logemann, and D. Loveland, "A Machine Program for Theorem-Proving," *Communications of the ACM*, vol. 5, no. 7, pp. 394–397, 1962, doi: 10.1145/368273.368557.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html